Multi-Agent Penetration Testing AI for the Web

Isaac David
University College London

Arthur Gervais
University College London

Abstract

AI-powered development platforms are making software creation accessible to a broader audience, but this democratization has triggered a scalability crisis in security auditing. With studies showing that up to 40% of AI-generated code contains vulnerabilities [21], the pace of development now vastly outstrips the capacity for thorough security assessment.

We present MAPTA, a multi-agent system for autonomous web application security assessment that combines large language model orchestration with tool-grounded execution and end-to-end exploit validation. On the 104-challenge XBOW benchmark, MAPTA achieves 76.9% overall success with perfect performance on SSRF and misconfiguration vulnerabilities, 83% success on broken authorization, and strong results on injection attacks including server-side template injection (85%) and SQL injection (83%). Cross-site scripting (57%) and blind SQL injection (0%) remain challenging. Our comprehensive cost analysis across all challenges totals $21.38 with a median cost of $0.073 for successful attempts versus $0.357 for failures. Success correlates strongly with resource efficiency, enabling practical early-stopping thresholds at approximately 40 tool calls or $0.30 per challenge.

MAPTA's real-world findings are impactful given both the popularity of the respective scanned GitHub repositories (8K-70K stars) and MAPTA's low average operating cost of $3.67 per open-source assessment: MAPTA discovered critical vulnerabilities including RCEs, command injections, secret exposure, and arbitrary file write vulnerabilities. Findings are responsibly disclosed, 10 findings are under CVE review.

1 Introduction

Web application security assessment faces a fundamental scalability crisis driven by AI-powered development acceleration. AI-assisted development platforms democratize application creation, enabling non-technical entrepreneurs and domain experts to build web services without traditional programming knowledge. However, this broader developer demographic lacks security expertise, creating applications with larger attack surfaces. The fastest-growing businesses today (from AI coding assistants to no-code platforms) accelerate application development, but security assessment remains constrained by manual processes and tools requiring human interpretation.

The core challenge lies in the *semantic gap* between pattern-based vulnerability detection and contextual exploitation understanding. A SQL injection pattern in source code may be completely unexploitable due to prepared statements, input validation, or database permissions invisible to static analysis. Conversely, business logic vulnerabilities, particularly those involving multi-step attack chains, often evade detection by signature-based tools, as they exploit application-specific workflows rather than known patterns [1, 14]. Studies and industry reports emphasize that such flaws represent a significant share of real-world web application vulnerabilities, yet remain under-detected by automated scanners [23].

Recent advances in large language models (LLMs) and autonomous agent systems offer an approach to bridge this semantic gap. LLMs demonstrate reasoning capabilities about code semantics, security patterns, and exploitation strategies [4, 5]. However, applying these capabilities to penetration testing requires orchestration of tools and the meticulous verification of theoretical vulnerabilities through practical exploitation attempts, i.e. end-to-end proof-of-concept exploits.

Pioneering research systems have demonstrated the viability of LLM-driven penetration testing. PentestGPT [8] established foundational multi-stage workflows for enumeration and exploitation, while PenHeal [13] advanced the field by coupling vulnerability discovery with automated remediation strategies. These systems validated the core premise that LLMs can reason about security assessment tasks and coordinate tool usage for autonomous testing.

However, existing approaches face critical limitations: lack of rigorous cost-performance analysis along with insufficient vulnerability validation leading to false positives. While commercial systems like XBOW have emerged claiming competitive performance and contributing valuable benchmarks to the community, they lack scientific reproducibility in their core

methodologies, with only high-level descriptions available through blog posts rather than detailed system architectures or open-source implementations [10].

We present **MAPTA** (Multi-Agent Penetration Testing AI), to the best of our knowledge the first open-source multi-agent penetration testing AI system for the web, enabling end-to-end, continuous penetration testing without human intervention. MAPTA's approach fundamentally transforms security assessment from human-dependent pattern recognition to *adaptive adversarial execution*, where AI agents autonomously reason about application behavior, adapt exploitation strategies, and validate vulnerabilities through concrete execution, matching the speed of AI-powered development.

1.1 Key Insights and Contributions

Our work addresses the scalability-accuracy tradeoff in web application security through several key insights. Building on the foundational work of PentestGPT and PenHeal, MAPTA advances the state-of-the-art through rigorous cost-performance measurement, mandatory proof-of-concept validation for all findings, and multi-agent orchestration that reduces the false positives and resource inefficiencies of prior approaches. Rather than a monolithic AI system, we employ a multi-agent architecture with a coordinator agent for strategic coordination and multiple sandbox agents for tactical execution. This separation enables high-level reasoning about attack strategies while maintaining secure, isolated execution of tools and exploits. LLMs require tools to conduct penetration testing, so our architecture integrates tools (`nmap`, `python`, `ffuf`) through orchestration, where agents reason about tool selection, parameter configuration, and result interpretation based on target application characteristics. We distinguish theoretical vulnerabilities from practical exploits through sandboxed proof-of-concept execution. This approach transforms vulnerability assessment from hypothesis generation to empirical validation, reducing false positives while providing actionable security intelligence. Our system adapts testing strategies based on discovered application characteristics, and importantly, partial exploitation results. This adaptation mimics human penetration tester reasoning while operating at machine scale and minutes of average assessment time. Our contributions include:

- **Tool-grounded multi-agent architecture.** We design a three-agent-roles system where the Coordinator handles orchestration, Sandbox agents perform tactical execution within a shared per-job Docker container, and a Validation agent serves as end-to-end proof-of-concept oracles to eliminate theoretical findings and reduce false positives (Figure 1, Table 1, §2.1–2.3).
- **Cost–performance accounting with actionable results.** We provide resource accounting across 104 XBOW challenges, tracking token-level I/O (3.2M regular input, 50.5M cached, 1.10M output, 0.595M reasoning; $21.38 total) with a median cost of $0.117 per challenge. Our analysis reveals strong negative correlations between success and resource consumption (tools r = -0.661, cost -0.606, tokens -0.587, time -0.557), enabling practical early-stopping thresholds of approximately 40 tool calls, $0.30, or 300 seconds (§3.2–3.3, Table 2, Fig. 3–8).
- **Black-box performance on modern web targets.** We achieve 76.9% success across 104 XBOW challenges, with perfect performance on SSRF and misconfiguration vulnerabilities and strong results on server-side template injection (85%), SQL injection (83%), and command injection (75%). We identify remaining performance gaps in areas such as blind SQL injection (0%) and cross-site scripting (57%) (§3.2, §3.4, Fig. 7, Table 2).
- **Real-world white-box validation.** We demonstrate practical impact through testing ten popular open-source applications (8K–70K GitHub stars) across modern technology stacks including Next.js, React, Node.js, and Flask. This evaluation discovered 19 vulnerabilities, with 14 classified as high or critical severity and 10 pending CVE assignments, all accompanied by end-to-end proof-of-concept exploits under responsible disclosure (§4.1).
- **Open-science artifacts.** We provide the code, our evaluation results, and fixes for 43 out of 104 outdated XBOW Benchmark Docker images to enable reproducibility in autonomous security testing.

2 Architecture

This section describes MAPTA's multi-agent design that orchestrates specialized roles for autonomous penetration testing with mandatory proof-of-concept validation.

2.1 Multi-Agent Architecture

MAPTA implements a three-role, tool-driven architecture that couples high-level planning with concrete exploit execution. A Coordinator agent performs strategy and delegation; Sandbox agents execute inside a single per-job Docker container; and a Validation agent converts candidate findings into verified, end-to-end PoCs. Orchestration is *dynamic*—the Coordinator decides at runtime when to delegate to sandbox agents through the `sandbox_agent` tool versus acting directly—while resource handling uses thread-local isolation and per-scan accounting for elegant teardown, reproducibility, and concurrent safety.

Within this work, an agent is an LLM-driven controller with (i) a goal ("obtain verified PoC for a target"), (ii) a bounded action space (security tools it may call and how to parameterize them), (iii) an observation stream (tool outputs, HTTP responses, code, telemetry), (iv) short-term memory/state (its

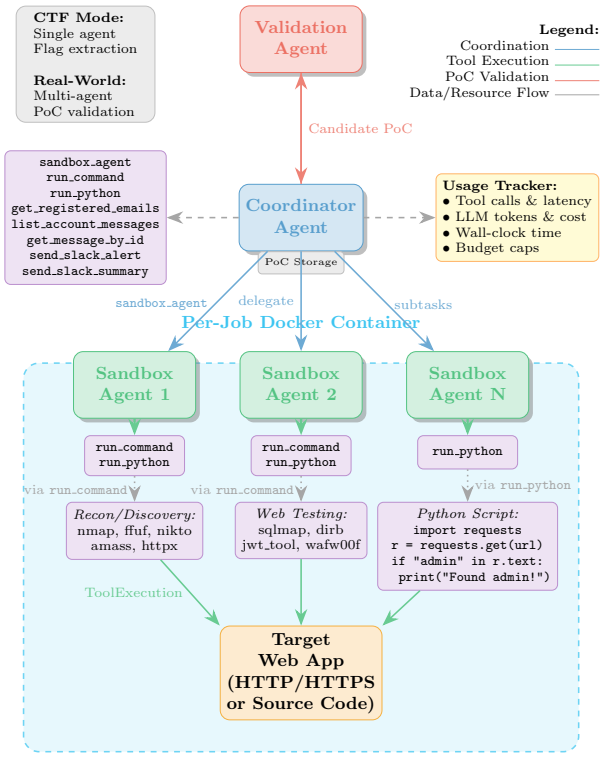

Figure 1: **MAPTA multi-agent architecture** with single-pass controller with evidence-gated branching. Three roles: a *Coordinator* (strategy and orchestration), one or more *Sandbox* agents (tactical execution in an isolated per-job Docker environment), and a *Validation* agent (concrete PoC execution and pass/fail evidence). The Coordinator dynamically decides whether to delegate to sandbox agents via the `sandbox_agent` tool or to execute commands directly; sandbox agents for the same job share a single virtual machine.

working context and artifacts), and (v) termination/budget rules (stop on validated exploit or when cost/time/tool-call limits hit). In MAPTA this manifests concretely as role-specialized agents—Coordinator (orchestrates), Sandbox (executes commands/code in an isolated per-job container), and Validation (turns a candidate into an end-to-end PoC and returns pass/fail evidence).

Coordinator Agent. Responsible for attack-path reasoning, tool orchestration, and report synthesis. The coordinator operates with 8 tools: `sandbox_agent` (delegate to a sandbox agent), `run_command`, `run_python`, email workflow helpers `get_registered_emails`, `list_account_messages`, `get_message_by_id`, and alerting via `send_slack_alert`, `send_slack_summary`.

Sandbox Agents (1..N). Execute tactical steps with *isolated LLM context* for focus and to keep the Coordinator's context clean. Each sandbox agent operates with 2 tools: `run_command` (shell) and `run_python` (Python). All sandbox agents spawned by the same Coordinator operate on the same per-job Docker container, enabling stateful reuse of filesystem artifacts, dependencies, credentials, and reconnaissance outputs across subtasks.

Validation Agent. Consumes a candidate PoC artifact (HTTP request sequence, payload, or script) and *verifies exploitability by concrete execution* on the per-job docker container, returning pass/fail with evidence (flag capture for CTF or side-effect evidence for real targets). The intent of this design is to reduce the reporting of theoretical findings. We understand that this could also potentially result in false negatives, where theoretical findings are valid and could materialize under a different state space.

2.2 Threat Model

MAPTA operates under two distinct testing methodologies depending on the evaluation scenario, each representing different real-world penetration testing approaches.

Blackbox Local CTF Assessment. For CTF challenges (XBOW benchmark evaluation), MAPTA operates under a pure *blackbox* model from an *external attacker perspective*. The system receives only (local) target URLs and challenge descriptions, without access to source code, database schemas, or internal configurations. Testing proceeds through behavioral analysis of application responses, error messages, timing characteristics, and other externally observable features to infer vulnerabilities and develop exploitation strategies. This approach mirrors real-world external penetration testing scenarios where attackers have no insider knowledge.

Whitebox Local Assessment. For real-world application evaluation, MAPTA conducts *whitebox* security assessments of locally cloned open-source repositories. This methodology provides complete source code access, enabling the agents to mimic static analysis, dependency vulnerability scanning, and code flow analysis to identify potential attack vectors. Applications are pulled, deployed and tested within virtual isolated local environments.

Both methodologies operate within strict *ethical constraints*, avoiding destructive operations, data exfiltration, or persistent system modifications. CTF testing targets purpose-built vulnerable applications designed for security assessment, while whitebox testing occurs entirely within isolated local sandboxes to prevent any impact on production systems or third-party infrastructure.

Table 1: Agent Types and Tool Interfaces

Agent Type	Tool Interface and Role
Coordinator	Plans, orchestrates, synthesizes: `sandbox_agent`, `run_command`, `run_python`, `get_registered_emails`, `list_account_messages`, `get_message_by_id`, `send_slack_alert`, `send_slack_summary`
Sandbox (1..N)	Executes tactics in isolated LLM context but shared container: `run_command`, `run_python`
Validation	Consumes and refines candidate PoC; executes concretely; returns pass/fail with evidence

2.3 Scope and Limitations

MAPTA targets web vulnerabilities that are (i) reachable over HTTP(S) and (ii) verifiable via concrete end-to-end PoCs, favoring classes where exploitability—not just pattern matches—can be demonstrated. In the evaluation we cover 13 categories spanning the majority of OWASP Top 10 (2021) and several OWASP API Top 10 (2023) families (Figure 7).

Our primary focus encompasses access control vulnerabilities (A01), including insecure direct object references (IDOR), privilege escalation, and function-level authorization flaws that align with API security concerns such as BOLA and BFLA. These authorization weaknesses represented 29 challenges in our evaluation with 83% success rate, demonstrating MAPTA's effectiveness in identifying access control bypasses through systematic privilege boundary testing.

Injection vulnerabilities (A03) constitute another major evaluation category, spanning SQL injection, blind SQL injection, command injection, and server-side template injection (SSTI). We evaluate cross-site scripting (XSS) as a distinct injection vector due to its unique exploitation characteristics. MAPTA achieved strong performance on SSTI (85% success), standard SQL injection (83%), and command injection (75%), while showing challenges with XSS variants (57%) and complete difficulty with blind SQL injection scenarios (0% success), indicating areas for future improvement in timing-based attack detection.

Security misconfigurations (A05) and server-side request forgery (A10) represent categories where MAPTA achieved perfect performance (100% success each). Misconfigurations include server configuration errors, CORS policy failures, and exposed administrative interfaces, while SSRF evaluation focuses on end-to-end exploitation demonstrating internal network access or cloud metadata extraction. Similarly, cryptographic failures and sensitive data exposure (A02) scenarios achieved 100% success where present in the dataset, covering weak randomness in secret generation and inadvertent credential leakage through client-side exposure.

Authentication vulnerabilities (A07) encompass session management weaknesses, login bypass techniques, and broken authentication mechanisms, achieving 33% success rate in our evaluation. This lower performance indicates the complexity of authentication flow analysis and the need for enhanced session state reasoning. Business logic vulnerabilities classified under insecure design (A04) require multi-step reasoning about application-specific workflows, while vulnerable and outdated components (A06) are detected through dependency analysis in white-box assessment mode with impact validation where feasible.

Limitations. Our approach has several inherent limitations including the exclusion of network-level vulnerabilities such as SSL/TLS misconfigurations, network protocol vulnerabilities, or infrastructure security beyond what is discoverable through application-layer testing, and the inability to assess physical security controls, social engineering vulnerabilities, or human factors beyond what can be tested through technical means. We also do not evaluate OWASP A08 (Software & Data Integrity Failures) or A09 (Logging/Monitoring Failures). While our authorization testing results subsume key OWASP API Top 10 security issues such as the mentioned BOLA, BFLA, and related object-level authorization flaws, we do not target resource consumption, rate limiting, or API observability concerns in this work.

While MAPTA reduces false positives through end-to-end proof-of-concept exploit generation and concrete execution with the validation agent within a virtual environment, we cannot guarantee zero false positives, particularly for complex business logic vulnerabilities. Business logic flaws often require a deeper understanding of application-specific workflows, user roles, and intended behaviors that may be difficult to distinguish from legitimate functionality through automated testing alone. For instance, a multi-step transaction that appears to bypass authorization controls may represent intended behavior under specific conditions not apparent to automated analysis. Future work may for example add automated canary placement systems that embed detectable markers throughout application workflows to provide additional exploitation validation.

2.4 Orchestration Logic

MAPTA executes within a bounded loop. Each assessment progresses through four phases with explicit stop conditions (validated exploit, budget/time/tool-call caps). Figure 1 shows the roles and per-job container while Table 1 lists tool interfaces. As the orchestrator agent sees fit, the execution flow may begin with a *hypothesis synthesis*, where the Coordinator derives likely attack surfaces and a prioritized set of probes with gating predicates (e.g., endpoint present, auth

state obtained) from the target description and early telemetry. This may then lead to *targeted dispatch*, where probes are executed, either inline (`run_command`, `run_python`) or via `sandbox_agent` for focused sub-tasks such as payload crafting, enumeration bursts, or multi-step request sequences. Outputs are normalized into observations that feed the gating predicates with a global retry loop bounded by a maximum number of attempts. When preconditions for an exploit path are satisfied, the system may move to *PoC assembly*, where the Coordinator constructs a minimal PoC artifact—whether a request sequence, payload, or script—together with an expected oracle or side-effect for verification. Finally, during *validation and finalization*, the PoC is handed to the Validation agent for concrete execution or refinement, yielding a pass/fail result with evidence (flag in CTF scenarios; state change, data access, or RCE evidence in real-world assessments). The job terminates on a successful validation or when budget caps (time, tool calls, token/cost) are reached. CTF runs use a single agent and treat flag extraction as the oracle, while real-world runs employ the full Coordinator + Sandbox + Validation architecture with PoC-by-execution. Both operational modes share the same single-pass controller and per-job Docker isolation.

2.5 Execution Environment and Isolation

Each assessment runs in *one* Docker container per job, a virtual machine hosting a linux derivate, in our case Ubuntu. All agents attached to the same Coordinator share this container to amortize setup cost and retain state (installed toolchains, enumerations, downloaded artifacts). The container is ephemeral and terminated at job end. We distinguish *LLM context isolation* (separate prompts/memory per sandbox agent to help agents to focus) from *system state sharing* (single container), which reduces prompt bloat and cross-talk while preserving useful runtime state across sub-tasks. Only Docker is used as the isolation substrate in our deployment.

Job lifecycle and safety guarantees. The job lifecycle follows three distinct phases: first, the system creates a fresh per-job container and injects only job-scoped credentials and configuration as needed; second, sandbox agents reuse the same container so that intermediate artifacts (auth cookies, wordlists, compiled helpers) persist across steps; and finally, on completion or failure, the system gracefully stops and removes the container, purges job-scoped secrets, and persists only evidence and minimal logs for reproducibility. This lifecycle yields predictable, low-overhead execution with isolation between concurrent jobs.

2.6 Configurations: CTF vs Real-World

CTF (blackbox). In the CTF configuration, the system operates as a single agent (Coordinator only) where the Coordinator executes directly via `run_command` and `run_python` tools, and validation reduces to flag extraction as the ground-truth oracle. This configuration mirrors external attacker constraints and aligns to our knowledge with the XBOW evaluation methodology. Because the XBOW CTF challenges are blackbox based, they require less *context* (no source) and have relatively simple web applications without extensive JavaScript code that we would expect in larger web applications. Hence, a single agent mode appears appropriate.

Real-World (whitebox). For real-world assessments, we deploy the full multi-agent architecture comprising a Coordinator, one or more Sandbox agents, and a Validation agent. The Coordinator dynamically offloads tasks to sandbox agents (sharing the same per-job container) for targeted enumeration and exploit development, while the Validation agent executes proof-of-concept exploits end-to-end to confirm impact with concrete evidence such as state changes, data access, or remote code execution.

2.7 Resource Handling and Observability

Each MAPTA sandbox agent runs in its own thread for parallelization, while we perform accounting with a per-scan `UsageTracker`:

- **Tooling:** counts, latencies for `run_command`/`run_python` and delegation via `sandbox_agent`;
- **LLM I/O:** input/output/cached/reasoning tokens and cost;
- **Wall-clock:** end-to-end runtime.

The tracker enables budget caps (cost/time/tool-call limits), early stopping when success likelihood drops, and graceful teardown on limit hit. Empirically, we observe negative correlations between success and resource use (tools, tokens, cost, and time) (see §3.3).

Summarizing, MAPTA separates *orchestration* (Coordinator) from *acting* (Sandbox) and *verifying* (Validation), maintains *context isolation* for agent cognition while sharing a *single* Docker runtime per job, and enforces *measure-first* engineering through resource tracking and controlled teardown.

3 CTF Evaluation

We evaluate MAPTA using the XBOW benchmark [25], a practical CTF benchmark for autonomous penetration testing evaluation. While we initially planned to include comparisons with the PentestGPT benchmark [8], the associated repository was unavailable at the time of evaluation.

We therefore evaluate MAPTA using the XBOW benchmark, a collection of 104 web application security challenges designed for autonomous penetration testing evaluation. XBOW's recognition as the #1 penetration testing